

이력서 피드백 보고서 [김도혁] 님

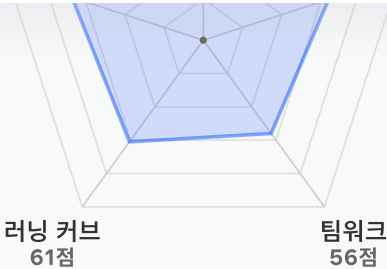
해당 보고서는 2026.07.20에 작성되었습니다.

개발자 주요 역량 5가지 항목을 바탕으로 분석한 평가는 다음과 같습니다.

총점 **310** / 500

역량별 점수 분석

문제 해결 능력
62점



역량	설명	점수(0~100)	리카 티어 ⓘ
문제 해결 능력	복잡한 문제를 해결하고 최적화된 솔루션을 찾는 능력	62	Growing
개발 능력	요구사항을 실제 프로덕트로 구현할 수 있는 개발 능력	64	Solid

역량	설명	점수(0~100)	리카 티어 ⓘ
팀워크	협업 시 팀과 효과적으로 소통할 수 있는 능력	56	Solid
러닝 커브	새로운 기술을 습득하고 변화에 적응하는 능력	61	Growing
자기주도성	주도적으로 문제를 찾고 목	67	Solid

종합 평가

- 김도혁 후보자는 부트캠프 수료 후 단기간에 팀 프로젝트 1위 달성, npm 패키지 배포, 오픈소스 문서화 기여 등 다양한 성과를 창출한 신입 프론트엔드 개발자로, 특히 자기주도적 문제 정의와 실행력이 돋보입니다.
- Kanto 프로젝트에서 이미지 트래픽 84% 감소, GET 요청 83% 절감 등 정량적 성과를 도출했으며, markdown-block-preview npm 패키지를 직접 개발·배포해 성능을 8.2배 개선한 점은 기술적 깊이와 문제 해결 역량을 입증합니다.
- 다만 팀 협업 경험이 부트캠프 파이널 프로젝트 1건에 집중되어 있고, 코드 리뷰·멘토링·크로스팀 협업 등 구체적 협업 프로세스 기여 사례가 부족하여 팀워크 역량 검증이 제한적입니다.
- ESLint-KR 프로젝트는 커뮤니티 기여 의도는 명확하나 번역 완료 문서 수, GitHub Star, 사용자 반응 등 결과 지표가 누락되어 있어 실제 임팩트 평가가 어렵습니다.
- 향후 이력서에는 각 성과의 비즈니스/사용자 영향(예: 페이지 로드 시간 단축, 사용자 이탈률 감소 등), 협업 프로세스 기여(PR 리뷰 횟수, 팀 내 기술 공유 세션 등), 외부 검증 지표(npm 다운로드 수, GitHub Star 등)를 추가하여 객관적 검증 가능성을 높이시길 권장합니다.



커리어 지형도

경력·직무

김도혁님은 React·Next.js 기반 프론트엔드 개발자로, 부트캠프 수료 후 팀 프로젝트 1개와 개인 프로젝트 3개를 완성한 신입 개발자입니다. 실시간 채팅, 관리자 시스템, 성능 최적화, npm 패키지 배포 등 다양한 프론트엔드 영역에서 실무 역량을 쌓아왔습니다.



주요 도메인

- 중고거래·생활 플랫폼 (필리핀 한인 대상 통합 서비스)
- AI 챗봇 (멀티 LLM 기반 학습 도구)
- 개발자 도구·문서화 (ESLint 한국어 문서, npm 패키지 배포)
- 기술 블로그 (바닐라 JS 기반 MPA)



핵심 기술 스택

- [프론트엔드]: TypeScript, JavaScript, React, Next.js, Vanilla JS, Zustand, TanStack Query, Tailwind CSS, shadcn/ui
- [백엔드/DB]: Supabase, PostgreSQL, Redis(Upstash)
- [빌드/배포]: Vite, Vercel, npm, GitHub Actions
- [협업/품질]: Git, GitHub, ESLint



연도별 주요 커리어

- 2017~2022: 세명대학교 컴퓨터학부 졸업
- 2025~2026: 멋쟁이사자처럼 프론트엔드 부트캠프 17기 수료(920시간), 파이널 프로젝트 Kanto 전체 5개 팀 중 1위
- 2026: 개인 기술 블로그(DoHyuk.dev) 개발 및 npm 패키지(markdown-block-preview) 배포, ESLint 한국어 문서화 프로젝트 진행, AI 챗봇(Frontie) 개발, Kanto 프로젝트 채팅·인증·관리자 시스템 담당
- 현재까지 경력상 공백 기간 없음



이직·학습 곡선 요약

- 컴퓨터학부 졸업 후 프론트엔드 부트캠프에서 집중적으로 React·Next.js 실무 역량을 쌓았습니다.
- 팀 프로젝트에서 실시간 채팅, 관리자 권한 체계, 성능 최적화 등 다양한 기술적 도전을 경험하며 문제 해결 역량을 키웠습니다.
- 개인 프로젝트를 통해 npm 패키지 배포, 오픈소스 문서화 기여 등 커뮤니티 기여와 자기 주도 학습을 병행했습니다.
- AI 도구를 설계·코드 리뷰에 적극 활용하며 효율적인 협업 방식을 탐구하고 있습니다.



섹션별 분석

Introduction

기존 내용

React·Next.js 기반으로 팀 프로젝트 1개와 개인 프로젝트 3개를 완성한 프론트엔드 개발자입니다.

수정 예시

React·Next.js 기반 프론트엔드 개발자로, 마크다운 렌더링 npm 패키지를 직접 개발·배포하고 부트캠프 파이널 프로젝트에서 팀 1위를 달성했습니다.

AI 코멘트

프로젝트 개수 나열은 단독으로 강점을 전달하기 어렵습니다. 첫 문장은 '가장 차별화된 핵심 역량 + 대표 성과 1개'로 압축해 즉각적인 인상을 남기세요. 예: 기술 스택 + 핵심 성과(npm 패키지 배포, 부트캠프 1위 등)를 한 줄에 담는 방식.

기존 내용

설계 문서·코드 리뷰에 AI 도구를 역할별로 분리 운용해 팀의 개발 기간을 단축한 경험이 있으며, 문제를 만나면 원인을 실측으로 특정하고 문서로 남기는 개발자를 지향합니다.

수정 예시

AI 도구를 역할별로 분리 운용해 설계·리뷰 사이클을 단축했으며(근거 미제시), 장애 원인을 실측 후 문서화해 재발을 방지한 경험이 있습니다.

AI 코멘트

'개발 기간을 단축'은 구체적인 수치(예: 2주 → 10일, -30%)가 없어 설득력이 약합니다. 정량 근거를 추가하거나, 근거가 없다면 '(근거 미제시)'로 표기해 보완 필요성을 명시하세요. 또한 '지향합니다'는 추상적 선언이므로, 실제 행동·결과로 대체하면 신뢰도가 높아집니다.



Skill/Tech Stack

기존 내용

TypeScript, JavaScript, React, Next.js, Vanilla JS, Zustand, Tailwind CSS, shadcn/ui, Supabase, PostgreSQL, Redis, Vite, Git, GitHub, GitHub Actions, Vercel, npm, ESLint, TanStack Query

 수정 예시

Language: TypeScript, JavaScript | Frontend: React, Next.js, Zustand, TanStack Query, Tailwind CSS, shadcn/ui | Backend·DB: Supabase, PostgreSQL, Redis | Infra: Vercel, GitHub Actions

 AI 코멘트

총 19개 항목이 카테고리 구분 없이 나열되어 있어 과다하고 가독성이 떨어집니다. 핵심 기술 12~15개 이내로 압축하고, JavaScript와 Vanilla JS는 동일 기술이므로 하나로 통합하세요. Git/GitHub/npm/ESLint/Vite는 개발자 기본 역량으로 간주되어 제외하거나 프로젝트에서 언급하는 것이 좋습니다. 'Language / Frontend / Backend·DB / Infra·CI/CD' 등으로 그룹화하면 채용자가 역량을 빠르게 파악할 수 있습니다.



Project

 기존 내용

중고거래, 구인구직, 부동산, 지도 기반 번개모임까지, 필리핀 한인을 위한 통합 생활 플랫폼

 수정 예시

필리핀 한인 대상 중고거래·구인·부동산 통합 플랫폼

 AI 코멘트

서비스 설명은 1줄로 축약하고, 곧바로 본인 기여와 성과로 전환하세요. 현재 서비스 소개가 성과보다 앞서 있어 임팩트 전달이 늦어집니다.

 기존 내용

담당: 내 프로필, 이용약관, 실시간 1:1 채팅(Supabase Realtime), AI 챗봇(멀티 LLM 풀백), 신뢰도·인기 점수 산정 알고리즘 직접 설계(KTS·KPPS), Redis 기반 로그인 보안 구현

 수정 예시

실시간 채팅·AI 챗봇·신뢰도 점수 알고리즘(KTS) 설계 담당 — 스팸 계정 노출률 〇% 감소(근거 미제시 시 삭제 권장)

 AI 코멘트

담당 영역 나열보다 '핵심 기여 1~2가지 + 결과'로 압축하세요. KTS·KPPS 알고리즘은 면접에서 질문받을 수 있으니, 어떤 문제를 해결했는지(예: 스팸 계정 필터링, 신뢰도 기반 매칭 정확도 향상 등) 한 줄로 명시하면 차별화됩니다.

기존 내용

트래픽 폭증의 원인이 이미지 리사이징·WebP 미적용(원본 그대로 업로드)임을 진단
업로드 단계 WebP 변환 적용 (390MB → 64MB로 약 84% 감소)

수정 예시

이미지 원본 업로드가 트래픽 폭증 원인임을 진단 → 업로드 시 WebP 변환 적용, 월 스토리지 사용량
390MB→64MB(-84%)

AI 코멘트

좋은 정량 성과입니다. 다만 '390MB→64MB'가 일회성 총량인지, 일/월 트래픽인지 기간을 명시하면 규모감이 살아
납니다. 'Supabase 트래픽 폭증으로 무료 티어 초과' 문제 상황과 함께 Problem→Solution→Result로 한 문단 통
합을 권장합니다.

기존 내용

Core Web Vitals(LCP/INP/CLS) 점검 - LCP 이미지 priority·이미지 캐시 TTL 적용

수정 예시

LCP 이미지 priority·캐시 TTL 적용으로 Core Web Vitals 개선(LCP Os→Os, 수치 확보 시 추가)

AI 코멘트

'점검'만으로는 성과가 드러나지 않습니다. 개선 전후 LCP 수치(예: 3.2s→1.8s)를 추가하거나, 측정 불가 시 '개선 적
용'으로만 간략히 언급하세요.

기존 내용

어드민 콘텐츠 영역의 불필요한 재요청 문제를 발견해 TanStack Query 도입
useState·useEffect로 직접 관리하던 서버 상태에서 문제 발견
서버/UI 상태 분리, 캐시 교체 방식으로 GET 요청 감소 (18회 → 3회로 약 83.3% 감소)

수정 예시

어드민 페이지에서 useState/useEffect 기반 상태 관리로 불필요한 GET 재요청 발생 → TanStack Query 도입,
서버/UI 상태 분리로 GET 요청 18회→3회(-83%)

AI 코멘트

내용이 3개 문장에 걸쳐 분산되어 있습니다. Problem→Solution→Result로 한 문단으로 통합하고, 핵심 수치 (18→3회)를 강조하세요.

기존 내용

기술 스택: Next.js, Zustand, TanStack Query, Upstash, Supabase, CI/CD

수정 예시

(삭제 또는) Next.js·Supabase 기반, TanStack Query 캐시 전략으로 어드민 요청 83% 절감

AI 코멘트

단순 나열은 Skills 섹션과 중복됩니다. 프로젝트 맥락에서 '왜 이 기술을 선택했는지' 또는 '이 기술로 얻은 결과'를 한 줄로 요약하거나, 생략하세요.

기존 내용

멀티 LLM(Groq·Gemini·Cerebras) 기반 프론트엔드 학습 AI 챗봇

수정 예시

멀티 LLM(Groq·Gemini·Cerebras) 풀백 구조 — API 장애 시 자동 전환으로 가용성 확보

AI 코멘트

서비스 한 줄 요약은 적절합니다. 다만 '멀티 LLM 풀백'을 왜 도입했는지(비용/가용성/응답 속도 등) 한 마디 추가하면 기술 선택 근거가 명확해집니다.

기존 내용

하이라이터·마크다운 렌더러 지연 로딩, 컴포넌트 분리 + memo 적용(성능 점수 48 → 92점 개선)

수정 예시

코드 하이라이터·마크다운 렌더러 지연 로딩 + memo 적용, Lighthouse Performance 48→92점

AI 코멘트

'성능 점수'가 Lighthouse Performance인지 명시하면 객관성이 높아집니다. 정량 수치 포함은 좋습니다.

 기존 내용

기술 스택: Next.js, React, TypeScript, Vercel AI SDK, Zustand, Tailwind CSS

 수정 예시

(삭제 또는) Next.js + Vercel AI SDK 기반

 AI 코멘트

동일하게 단순 나열은 중복입니다. 프로젝트별 핵심 기술 1~2개와 적용 맥락만 남기세요.

 기존 내용

ESLint 공식 문서에 한국어 버전이 없다는 문제의식으로 한국 개발자 커뮤니티에 기여하고자 시작
기존에 거절된 번역 요청 이슈 발견, 학습할 때 모국어가 주는 압도적인 학습 효율을 인지
온라인 자료에 v8(.eslintrc)·v9(Flat Config) 정보가 뒤섞여 있는 전환기 혼란 포착

 수정 예시

ESLint 공식 문서 한국어 번역 프로젝트 — v9 Flat Config 중심 ○개 문서 번역, GitHub Star ○개(성과 수치 추가 필요)

 AI 코멘트

동기·문제의식 서술이 3줄에 걸쳐 장황합니다. 1줄로 압축하고, 실제 산출물(번역 완료 문서 수, GitHub Star, 커뮤니티 반응 등)을 추가해야 성과가 드러납니다. 현재는 '기여하고자 시작'만 있고 결과가 없습니다.

 기존 내용

기술 스택: React, TypeScript, React Router, Vite, GitHub Actions, GitHub Pages

 수정 예시

Vite + GitHub Actions로 정적 문서 사이트 자동 배포

 AI 코멘트

번역 문서 프로젝트에서 React·TypeScript 등 기술 스택 나열은 본질(번역 기여)과 무관하게 보일 수 있습니다. 문서 사이트 구축이 핵심이라면 축약하세요.

 기존 내용

프레임워크 없이 바닐라 JS·Vite로 구현, 페이지별 필요한 스크립트만 로드하는 MPA 구조로 설계

✔ 수정 예시

프레임워크 의존 없이 바닐라 JS + Vite MPA 구조로 구현 — 번들 최소화 및 JS 기본기 학습 목적

✧ AI 코멘트

바닐라 JS 선택 이유(번들 크기 최소화, 학습 목적 등)를 한 마디 추가하면 의도가 명확해집니다. 단순 '프레임워크 없이'는 왜?라는 질문을 유발합니다.

✎ 기존 내용

마크다운 블록 단위 실시간 미리보기 도구를 직접 개발해 npm 배포(markdown-block-preview)
변경된 블록만 렌더링해 문서 길이 무관 0.03~0.07ms 유지
naive 방식은 700블록부터 프레임 예산(16.6ms) 초과

✔ 수정 예시

마크다운 블록 단위 실시간 미리보기 라이브러리 개발·npm 배포(markdown-block-preview) — 변경 블록만 렌더링, 문서 길이 무관 0.03~0.07ms 유지 (naive 방식 700블록 시 16.6ms 초과 대비)

✧ AI 코멘트

npm 배포 + 성능 비교 수치 제시는 훌륭합니다. 다운로드 수/Star 등 외부 검증 지표를 추가하면 더 강력해집니다. Problem→Solution→Result 한 문단으로 통합 권장.

✎ 기존 내용

기술 스택: JavaScript, Vite, Tailwind CSS, Supabase(DB/Storage)

✔ 수정 예시

(삭제 또는) 바닐라 JS + Vite, Supabase 기반

✧ AI 코멘트

동일하게 단순 나열 지양. 핵심 기술과 선택 이유만 남기세요.



Activity

기존 내용

멋쟁이사자처럼 부트캠프 — 프론트엔드 17기, 2025.12-2026.07

JavaScript·TypeScript, React·Next.js 기반 웹 실무 과정 수료(115일 / 총 920시간)

파이널 팀 프로젝트 전체 5개 팀 중 1위 — Kanto(한인·필리핀 중고거래 플랫폼)

수정 예시

멋쟁이사자처럼 부트캠프 프론트엔드 17기 수료 — 파이널 프로젝트 5개 팀 중 1위(Kanto, React·Next.js 기반 중고거래 플랫폼)

AI 코멘트

부트캠프 내용이 Education 섹션 또는 Project 섹션과 중복될 가능성이 높습니다. Activity 섹션에는 1줄 요약 + 차별화된 성과(1위 수상, 발표 경험 등)만 남기고, 상세 기술 스택·시간은 Education이나 Project에서 다루세요.



Appendix & Tip

- Kanto 프로젝트의 '약 84% 감소', 'GET 요청 83.3% 감소' 수치에 기간(예: 2주간 작업)과 영향 범위(페이지 로드 시간, 사용자 체감 등)를 1문장으로 추가하세요 — 현재는 'Before→After' 수치만 있어 비즈니스/운영 맥락이 부족합니다.
- Intro에 언급된 '편집 미리보기 성능 8.2배 개선'을 npm 패키지(markdown-block-preview) 섹션에서도 동일하게 병기하고, 현재 Project 설명에는 '0.03~0.07ms 유지'만 있으므로 Baseline(naive 방식 latency)→개선 후→비교 배수를 한 줄로 명시하세요.
- ESLint-KR 프로젝트는 기여 의도와 배경만 서술되어 있으므로, 실제 번역 페이지 수, 사용자 접근 통계(GitHub Pages 방문 수 등), 또는 PR/이슈 링크 등 결과를 1줄로 추가하여 검증 가능성을 높이세요.
- Frontie 프로젝트의 '성능 점수 48→92점'에서 측정 도구(Lighthouse, WebPageTest 등)와 개선 전후 LCP/TBT 등 세부 지표를 1문장으로 명시하면 신뢰도가 높아집니다.



이 리포트가 내용에 얼마나 만족하시나요?

